

ICS Lab1: Data Lab 实验报告

姓名: 南涵译

学号: 25300730034

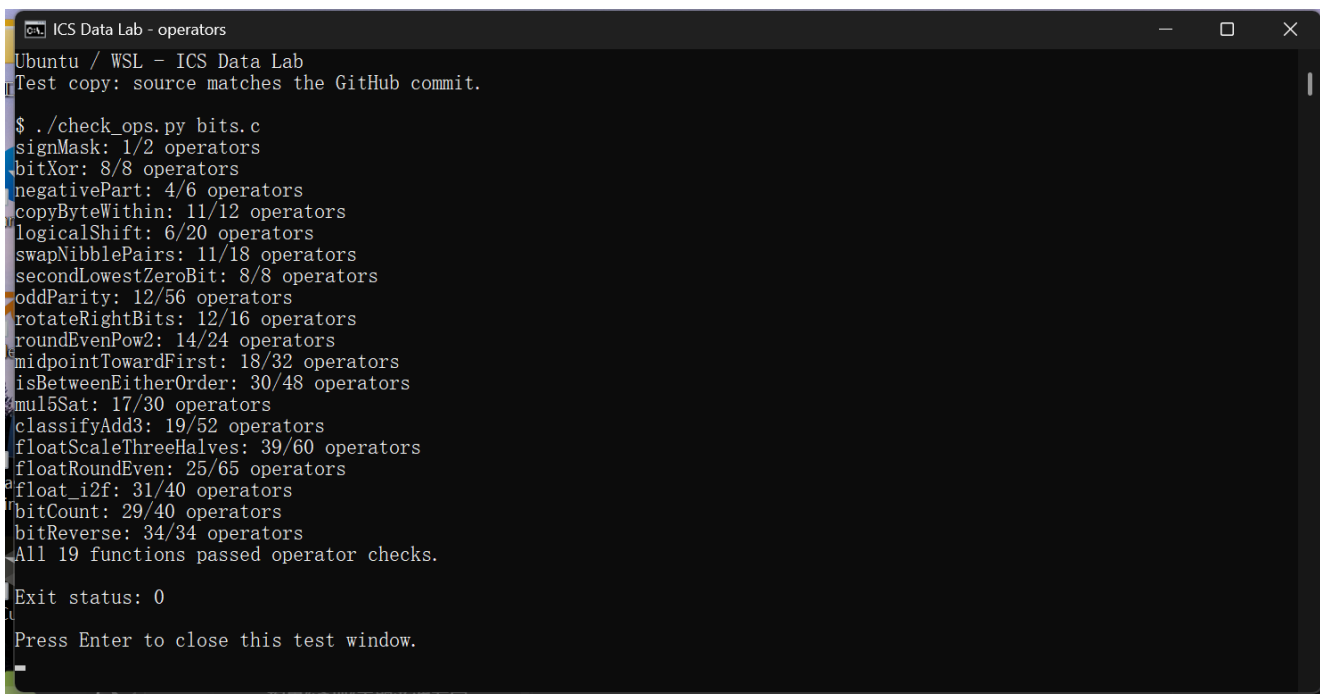
实验内容

在 32 位补码模型及规定运算符、常量和操作数限制内实现 bits.c 的 P1~P19。浮点题直接处理 IEEE 754 单精度的位模式，不使用浮点类型或运算。

测试在 Ubuntu / WSL 的隔离副本中执行，副本 bits.c 与已经推送到 GitHub 的代码内容一致。以下为实际终端窗口截图。

运算符检查

执行 ./check_ops.py bits.c, 19 个函数均通过规则检查。



```
ICS Data Lab - operators
Ubuntu / WSL - ICS Data Lab
Test copy: source matches the GitHub commit.

$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 11/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 11/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 14/24 operators
midpointTowardFirst: 18/32 operators
isBetweenEitherOrder: 30/48 operators
mul5Sat: 17/30 operators
classifyAdd3: 19/52 operators
floatScaleThreeHalves: 39/60 operators
floatRoundEven: 25/65 operators
float_i2f: 31/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.

Exit status: 0

Press Enter to close this test window.
```

19题运算符检查全部通过

正确性测试

执行 `make clean`、`make all` 后运行 `./btest`，19 题 Errors 均为 0，总分为 110/110。

```
ICS Data Lab - correctness
eUbuntu / WSL - ICS Data Lab
Rebuilt the current source with make clean and make all.

$ ./btest
Score  Rating  Errors  Function
1      1        0      signMask
2      2        0      bitXor
3      3        0      negativePart
4      4        0      copyByteWithin
4      4        0      logicalShift
4      4        0      swapNibblePairs
4      4        0      secondLowestZeroBit
5      5        0      oddParity
5      5        0      rotateRightBits
5      5        0      roundEvenPow2
5      5        0      midpointTowardFirst
7      7        0      isBetweenEitherOrder
7      7        0      mul5Sat
7      7        0      classifyAdd3
7      7        0      floatScaleThreeHalves
10     10        0      floatRoundEven
10     10        0      float_i2f
10     10        0      bitCount
10     10        0      bitReverse
Total points: 110/110

Exit status: 0
Press Enter to close this test window.
```

正确性测试110分全部通过

各题实现思路

P1 signMask

1 的二进制表示只有最低位为 1，左移 31 位后，这个 1 就到了最高位，其余位都是 0。结果对应 `0x80000000`，在有符号 `int` 中解释为 `INT_MIN`；题目要的是这个位模式，不是正数 2 的 31 次方。只需要一次左移运算。

P2 bitXor

异或的结果只在两个对应位不同的时候为 1，所以分别提取“x 为 0、y 为 1”和“x 为 1、y 为 0”的位置，得到 `~x & y` 和 `~y & x`。两部分本来可以直接做或运算，但本题只允许 `~` 和 `&`，因此用德摩根律，先分别取反、做与运算，再把结果整体取反。这里判断的是每一位，不需要比较 x、y 的大小；共用 8 次运算。

P3 negativePart

算术右移 `x >> 31` 可以得到符号掩码：非负数得到全 0，负数得到全 1。把掩码与 x 相与，就能清除非负数、保留负数，之后统一取反加 1，得到需要的相反数，整个过程不使用 `if`。`INT_MIN` 的正数相反值超出 `int` 范围，这里按实验的位向量规则保留低 32 位，结果仍是 `INT_MIN` 的位模式。

P4 copyByteWithin

一个字节占 8 位，用 `src << 3`、`dst << 3` 算出两个字节的位偏移。先把来源字节右移到最低 8 位，再与 `0xFF` 相与，清除算术右移可能补出的高位；这样取出的来源数据可以安全地左移到目标位置。

目标位置的旧内容用单独的目标掩码取出，取反加 1 表示它的相反数，再从 x 中减去并加上新内容。其他字节不变。先统一到最低位，也避免了按 `dst` 与 `src` 的位置差移动时出现负移位的问题；两者相同时，减去和补回的是同一个字节。

P5 logicalShift

普通的 int 右移会复制符号位，负数左侧补 1，所以先算术右移，再用掩码清除补出来的高位。以 $1 \ll 31$ 为起点，右移 n 位得到高 $n+1$ 位为 1 的数，左移一位后取反，就得到“高 n 位为 0，其余位为 1”的掩码。

$n=0$ 时，最高位左移出界，取反后得到全 1，保留原数； $n=31$ 时，掩码只剩最低位为 1，保留原来的最高位。这样不需要额外分支，也不会出现移动 32 位的情况。

P6 swapNibblePairs

逐字节拆开处理会重复生成掩码和移动数据，运算次数较多。改为把 $0x0F$ 扩展成 $0x0F0F0F0F$ ，一次选中四个字节的低半字节。低半字节统一左移 4 位，高半字节则把 x 右移 4 位后用同一掩码取出。两部分互不重叠，相加就能合并，并保持各字节原来的位置。

P7 secondLowestZeroBit

先令 $u = \sim x$ ，把找第二个最低的 0 转换成找第二个最低的 1。 $u \& (\sim u + 1)$ 只保留最低的 1：取反加 1 后，它以下的位仍为 0，它以上的位与原数互补，因此做与运算后只留下这个位置。

得到第一个标记后，用异或把它从 u 中清除，再对剩余数重复一次最低 1 的提取。若原数没有两个 0，剩余数就是 0，最后也会返回 0，无需另外判断。

P8 oddParity

异或可以记录 1 的个数是奇数还是偶数，因为两个 1 异或会抵消。依次把 x 右移 16、8、4、2、1 位并与原来的 x 异或，把各部分的奇偶性逐步折叠到最低位，不必逐位计数。最低位为 1 表示原数有奇数个 1，而题目要求偶数个时返回 1，因此最后对最低位做逻辑取反。

P9 rotateRightBits

循环右移需要把低位移出的内容补到高位，而不是直接丢掉。先用 $n \& 31$ 取出低 5 位，相当于把旋转位数对 32 取余；右边部分采用算术右移后加掩码的办法，左边部分用左移补回，最后做或运算。

左移位数也与 31，把“32 减去旋转位数”限制到 0~31。旋转 0 位时，两部分都是原数，做或仍得到原数，因此不用特殊分支，也避免了左移 32 位。

P10 roundEvenPow2

清除低 n 位可以得到 2 的 n 次方的倍数，所以把问题转为“先加一个合适的偏移量，再清除低位”。偏移量取半个间隔减 1：超过中点的数会进到上一个倍数，不到中点的数仍留在下一个倍数。

恰好位于中点时，看原商的奇偶性。原商为奇数，偏移量再加 1，使结果进到偶数商；原商为偶数，则保持较小的倍数。例如 $n=2$ 时，10 位于 8 与 12 之间，选 8；14 位于 12 与 16 之间，选 16。

P11 midpointTowardFirst

直接计算 $x+y$ 可能溢出，改用 $(x \& y) + ((x \oplus y) \gg 1)$ 。相同的 1 位由与运算提取，不同的位由异或提取，相当于先分开整理再除以 2，得到向下取整的中点。

若 x 、 y 的和为奇数，中点位于两个整数之间；只有 x 较大时需要把结果加 1，向第一个参数靠近。因此 (4,7) 得到 5，交换参数后得到 6。比较大小时，异号直接看符号，同号才看差值的符号，避免差值溢出造成误判。

P12 isBetweenEitherOrder

分别判断 x 是否小于 a 、是否小于 b 。如果两个判断不同，说明 x 在两个端点之间，不需要先给端点排序。再把 x 等于 a 或 b 的情况加入，就能覆盖闭区间，也能处理两个端点相等的情况。

比较时不能对所有输入直接看 $x-a$ 的符号，因为异号相减可能溢出。先检查符号是否不同：异号时由符号决定大小，同号时才用差值的符号判断。最后用两次逻辑取反把结果统一成 0 或 1。

P13 mul5Sat

用加法依次得到 $2x$ 、 $4x$ ，再加 x 得到 $5x$ 。不能只检查最后一次结果的符号，因为中间可能已经溢出并发生回绕。因此把各阶段与 x 的符号变化合并，只要有一步发生变化，就使用饱和值。

原数非负时，饱和值取 INT_MAX ；原数为负时，取 INT_MIN 。溢出标记扩展成全 0 或全 1 的掩码，分别选择普通乘法结果或边界值，全程不使用条件语句。

P14 classifyAdd3

把三数相加拆成 $x+y$ 和“第一次结果加 z ”。每次用“两个加数同号、结果却异号”检测有符号溢出。正溢出记为 1，负溢出记为 -1，没有溢出记为 0，把两次修正相加，就得到真实总和是否超出范围的判断。

关键是保留两次溢出的方向，而不是只要出现过溢出就认定最后越界。比如两个 INT_MAX 先相加发生正溢出，再加 INT_MIN 后，真实总和重新落入 int 范围，两次修正正好抵消，返回 0。

P15 floatScaleThreeHalves

先拆出符号、阶码和尾数字段。规格化数补上省略的最高位 1，用整数计算有效数乘 3，再根据结果是否需要重新规格化，选择右移 1 位或 2 位并调整阶码。只在最终右移时舍入一次，避免先后舍入带来的误差。

舍弃部分大于一半就进位，等于一半时只在保留部分为奇数时进位，实现最近偶数舍入。非规格化数没有隐藏的 1，单独处理，并允许结果进入规格化范围。正负零保留符号，NaN 和无穷原样返回；阶码超出有限数范围时返回对应符号的无穷。

P16 floatRoundEven

根据阶码确定哪些位属于小数部分，先清除这些位，再比较舍弃部分与半个单位。大于一半时进位，恰好一半时看整数部分的奇偶性，保证选中的整数为偶数，例如 1.5 变成 2，2.5 也变成 2。

绝对值小于 0.5 的数返回带原符号的零；恰好 0.5 也舍入到零。阶码足够大时，单精度数已经没有可表示的小数位，直接保留。NaN、无穷同样原样返回。最终返回的是舍入后整数的浮点位模式，而不是 int 类型的整数值。

P17 float_i2f

先分离符号，再用 unsigned 保存绝对值。 INT_MIN 的正数相反值无法用 int 表示，但用 unsigned 取反加 1 可以得到它的幅值。找到最高的 1 所在位置后，就能确定阶码，再把有效数对齐到包含隐藏位的 24 位。

有效位较少时左移即可，较多时右移并检查舍弃部分，按最近偶数规则进位。如果进位使有效数多出一位，再右移一次并增加阶码。 $x=0$ 单独返回 0，避免找不到最高有效位。

P18 bitCount

用 `0x55`、`0x33`、`0x0F` 扩展出分组掩码。先统计每 2 位中 1 的数量，再合并为每 4 位、每 8 位的数量；每一步都把相邻两组对齐后相加，一次处理整个 32 位数。

最后通过右移 8 位、16 位继续合并计数。结果范围是 $0 \sim 32$ ，6 位足以表示，因此最后与 `0x3F`，清除高位残留的分组信息，只保留总数。

P19 bitReverse

把整体反转拆成五次交换：先交换相邻的 1 位组，再交换 2、4、8、16 位组。每一步用掩码取出两边的数据，分别移到对方的位置后做或运算；右移部分加掩码，清除可能补出的符号位。

本题最多允许 34 次运算，因此掩码也要复用。先从 `0xFF` 构造低 16 位为 1 的掩码，再通过移位与异或逐级得到字节、半字节、2 位和 1 位掩码，减少重复拼接的次数。五次交换完成后，第 i 位就被移到第 $31-i$ 位。

参考资料

- [课程 Lab1 实验说明](#)。
- 实验仓库 `README.md`、`bits.c` 中的题意和编码规则，以及各题的实际实现。